Master BA Interview Preparation Guide (13-Level Framework) — WariMitra

Document Identifier: WM-INT-PREP-024

Classification: 13-Level Technical Business Analyst Interview Question & Answer Bank

Repository Basis: Grounded directly in `WariMitra` codebase evidence

Target Audience: BA / Technical BA / Agile Product Owner Candidates

Level 1: Resume Verification & Project Introduction

Q1.1: Walk me through this project listed on your resume: WariMitra.

Short Answer: WariMitra is a unified GovTech digital ecosystem engineered to manage Maharashtra's 700-year-old Pandharpur Wari pilgrimage—a mass gathering of 1M+ devotees walking 250 km over 21 days.

** Detailed Answer: The project combines a multilingual Flutter mobile app for pilgrims and field volunteers with a Next.js 15 command center for district police and medical authorities. It replaces manual paper registers and fragmented VHF radio calls with sub-2-second emergency SOS dispatch, Geohash proximity medical searches, real-time temple queue telemetry, and an asynchronous Web3 trust layer anchoring emergency incident snapshots to Ethereum Sepolia at zero gas cost.*

** Project Evidence:** `README.md:1-65`, `ARCHITECTURE.md:1-59`.

Level 2: Project Architecture & Scope

Q2.1: What was the architectural rationale for migrating from Django to Firebase?

Short Answer: Django's synchronous database connection model would have suffered fatal transaction lockups under concurrent GPS pings from 500k+ pilgrims.

** Detailed Answer: In our architectural review (`backend_audit.md`), we determined that handling 50,000+ live GPS pings per minute in a single PostgreSQL instance would exhaust connection pools and lock tables. We migrated to Firebase Serverless (Cloud Firestore in `asia-south1` and Cloud Functions v2). This provided elastic auto-scaling to 10,000+ concurrent connections, native real-time listeners for the command center, offline mobile persistence, and cut projected infrastructure costs to ~$185/month for 1M users.*

mobile persistence, and cut projected infrastructure costs to ~$185/month for 1M users.

** Project Evidence:** `Reference files/backend_audit.md:1-81`, `ARCHITECTURE.md:452-463`.

Level 3: Core BA Fundamentals

Q3.1: How do you differentiate a Business Requirement from a Functional Requirement in WariMitra?

Short Answer: A Business Requirement states the high-level operational objective; a Functional Requirement specifies the exact system behavior, input, and response.

** Project Example:*

- Business Requirement (`BR-001`): *The system must reduce fatal emergency response times by providing an immediate distress channel for pilgrims.*

- Functional Requirement (`FR-001`):* The mobile client shall require a continuous 3.0-second hold gesture on the SOS button, acquiring device GPS, generating a UUIDv4 idempotency key, and calling `createSOS` to persist an alert in Firestore with status `pending`.

Level 4: Technical BA Competencies

Q4.1: How does WariMitra query nearby medical camps without using an expensive GIS database like PostGIS?

Short Answer: It uses Geohash bounding-box range queries in Cloud Firestore combined with server-side Haversine distance filtering.

** Detailed Answer: The Cloud Function `getMedicalCamps` computes Geohash bounding boxes for the user's location and a 10 km radius using `geofire-common`. It runs parallel Firestore queries (`orderBy('geohash').startAt().endAt()`) against camps marked `status == 'active'`. It then computes the Great-Circle Haversine distance, filters out camps > 10 km, and sorts the qualifying camps in ascending distance order, returning in 320ms p95.*

** Project Evidence:** `functions/src/index.ts:122-177`.

Level 5: Scenario-Based Questions

Q5.1: What happens if an elderly pilgrim panics and presses the SOS button 10 times in 30 seconds?

Short Answer: The system gracefully absorbs the traffic using idempotency and rate limiting, creating only 1

Short Answer: The system gracefully absorbs the traffic using idempotency and rate limiting, creating only 1 alert and preventing responder congestion.

** Detailed Answer: The client UI requires a 3-second hold to trigger. If the user repeatedly triggers it, the first call creates the incident. Subsequent retries with the same `idempotencyKey` return the existing `alertId`. If distinct calls are made, the backend rate limiter checks the user's history and blocks requests beyond 5 per minute with HTTP 429 (`resource-exhausted`), showing a polite Marathi cooldown message on the phone.*

** Project Evidence:** `functions/src/index.ts:43-70`.

Level 6: Metrics & KPI Audit

Q6.1: You claim a 1.2-second SOS creation latency. How was that verified versus the 50% response time reduction?

Short Answer: 1.2s is a verified p95 technical benchmark measured under load testing; 50% is a strategic operational target.

** Detailed Answer: We evaluated the 95th percentile Cloud Function execution and database persistence latency under network load tests, measuring 1.2s (`README.md:243`). Conversely, the 50% ambulance arrival reduction is a target modeled against estimated historical baseline response times (45 minutes down to 20 minutes). We cannot mathematically certify the 50% claim from repository code alone because pre-digital dispatch logs do not exist in the codebase.*

** Project Evidence:** `14_METRICS_AUDIT/METRICS_AUDIT.md`.

Level 7: Stakeholder Management & Conflict

Q7.1: How did you resolve the conflict between police demanding strict Aadhaar KYC and pilgrims needing zero-friction SOS?

Short Answer: Decoupled identity verification from emergency triage: allow immediate anonymous/phone distress triggers, while restricting authority actions to verified credentials.

** Detailed Answer: Police initially wanted full biometric registration to prevent prank calls. However, requiring elderly rural pilgrims to complete Aadhaar eKYC while suffering chest pain would cause fatal drop-offs. As the BA, I proposed a compromise: keep the SOS panic trigger friction-free (authenticated via basic phone OTP, secured by rate limiting and geofencing), while enforcing strict RBAC custom claims on the responding authority side.*

authority side.

** Project Evidence:** `firestore.rules:26-28,54-56`.

Level 8: Agile & Scrum Delivery

Q8.1: How did you slice the Emergency SOS feature into incremental user stories?

Short Answer: Sliced vertically by risk: core ingestion first, then anti-abuse validations, real-time push, and finally Web3 audit proofing.

** Detailed Answer:*

- Sprint 1: *Basic 3-second hold button and Firestore write (`US-SOS-001`).*
- Sprint 2: *Backend validation pipeline—idempotency UUIDs and 5/min rate limiting (`US-SOS-002`, `US-SOS-003`).*
- Sprint 3: *Asynchronous FCM multicast push notifications to responder roles (`US-CMD-001`).*
- Sprint 4:* Asynchronous 0-ETH Sepolia blockchain snapshot anchoring (`US-W3-001`).

Level 9: SQL, NoSQL & Data Modeling

Q9.1: Why did you choose NoSQL document collections over relational SQL tables for WariMitra?

Short Answer: Firestore NoSQL provides horizontal auto-scaling, direct real-time snapshot listeners, and native mobile offline caching.

** Detailed Answer: A relational database requires connection pooling (PgBouncer), schema migrations, and complex WebSocket fan-out layers (Redis/Celery) that fail under 100k+ concurrent mobile connections. Firestore provides native real-time listeners (`onSnapshot`) that sync web command centers in 1.5s p95, handles offline document caching on Flutter devices, and auto-scales writes to 10,000/sec without manual sharding.*

** Project Evidence:** `ARCHITECTURE.md:188-263`.

Level 10: API Design & Architecture

Q10.1: Why did you use HTTPS Callable Cloud Functions instead of raw Firestore direct writes for SOS?

Short Answer: To enforce server-side business rules (rate limiting, geofencing, idempotency) that cannot be safely entrusted to client devices.

** Detailed Answer: If mobile clients wrote directly to the `sosAlerts` collection, a compromised or malicious*

** Detailed Answer: If mobile clients wrote directly to the `sosAlerts` collection, a compromised or malicious client could bypass rate limiting, forge timestamps, or spoof coordinates. By channeling distress requests through the `createSOS` Callable Function, the server enforces strict pre-ingestion validation, sanitizes inputs, calculates Geohashes, and generates immutable audit logs before persisting the document.*

** Project Evidence:** `functions/src/index.ts:20-116`.

Level 11: UAT & Quality Assurance

Q11.1: How did you design acceptance criteria for low-literacy rural users?

Short Answer: Used Gherkin behavioral specifications focused on physical gestures, audio-visual feedback, and zero typed text.

** Detailed Answer:** We established acceptance criteria verifying that the UI displays large Marathi typography, high-contrast red panic elements, continuous radial progress rings during touch, and haptic buzzes. Our UAT scripts (`17_UAT/UAT_TEST_CASES.md`) explicitly tested that releasing the hold gesture before 3.0 seconds cleanly resets progress without false-triggering network packets.

Level 12: Behavioral & Situational (STAR)

Q12.1: Tell me about a time you identified a critical flaw before production.

Situation: During pre-production review of the initial prototype, the backend relied on Django 6.0 with synchronous database writes for GPS pings.

** Task: As the BA, I evaluated whether this would satisfy our NFR of supporting 1M+ pilgrims walking simultaneously.*

** Action: I reviewed the technical audit (`backend_audit.md`), facilitated an architectural workshop, and demonstrated that synchronous writes would lock the database. I authored the business justification for transitioning to Firebase Serverless.*

** Result:** Successfully pivoted to Firebase v2.0, achieving 1.2s p95 SOS latency, 320ms read latency, and reducing projected infrastructure costs to ~$185/month.

Level 13: Challenging Cross-Questions

Q13.1: If someone says: 'Blockchain is completely unnecessary hype for an emergency app'—how do

Q13.1: If someone says: 'Blockchain is completely unnecessary hype for an emergency app'—how do you defend the Web3 Trust Layer?

Short Answer: It solves the specific legal problem of non-repudiation and post-disaster audit integrity at ₹0 operational cost without touching the real-time emergency path.

** Detailed Answer:**

> *"I completely agree that putting real-time emergency triage on a blockchain would be catastrophic due to gas fees and latency. That is why WariMitra does NOT run operations on blockchain. 100% of emergency operations run on high-speed Firebase (1.2s p95). The Web3 layer is strictly an asynchronous, background audit notary. By packing SHA-256 event snapshots into 0-ETH transactions on Sepolia, we provide an immutable, mathematically unalterable log for post-event judicial inquiries, eliminating government liability disputes at exactly ₹0 cost to the state."*

* **Project Evidence:** `docs/WEB3_ARCHITECTURE.md:1-49`, `functions/src/trustLayer/`.